Feature Engineering — Making Your Data Smarter

By the end of today you will understand why feature engineering is the highest-leverage activity in ML, and master the core techniques: handling missing values professionally, encoding categorical variables, scaling and transforming numeric features, creating interaction and polynomial features, extracting features from dates, and engineering features from domain knowledge. A well-engineered dataset with a simple model almost always beats a raw dataset with a complex model.

The most important mindset shift in Phase 2:

Days 13-20: 'Which model should I use?'
Days 21-23: 'How can I make my data better?'

In practice, 80% of performance gains come from better features, not better algorithms. The best Kaggle competitors spend most of their time on feature engineering, not model selection.

SECTION 01 What is Feature Engineering?

Feature Engineering

What it is: The process of using domain knowledge and data transformations to create, modify, or select input features that make ML models more accurate. It is the bridge between raw data and a trained model. Good features make even simple models powerful. Poor features make even complex models weak.

Real-life analogy: Imagine you want to predict if someone will buy a house. The raw data has: date of birth (1985-03-15), purchase date (2024-07-01). These raw columns tell the model nothing useful directly. But if you engineer: age=39, is_weekend_purchase=False, month=7, age_group='35-45' — suddenly the model has meaningful signals to work with.

In Python:
```
df['age'] = (pd.Timestamp.now() - df['birth_date']).dt.days // 365
```

Why feature engineering is the most important ML skill:

Experiment 1: Raw data + XGBoost -> 82% accuracy
Experiment 2: Engineered features + XGBoost -> 91% accuracy
Experiment 3: Engineered features + Logistic Regression -> 88% accuracy

> Adding 10 good features beat switching to a more powerful algorithm.
> And those engineered features worked with EVERY model, not just one.
>
> Features encode human knowledge about the problem.
> Models learn patterns from data. You give them better signals to learn from.

Feature Engineering Technique	What It Addresses
Handling missing values	Imputation strategies beyond simple mean/median fill
Encoding categorical variables	Converting text categories to numbers models can use
Scaling and transformations	Making numeric distributions model-friendly
Date and time features	Extracting day, month, hour, weekday from timestamps
Interaction features	Combining two features to capture their joint effect
Polynomial features	Capturing non-linear relationships for linear models
Binning / discretisation	Converting continuous to categorical where appropriate
Domain-specific features	Using expert knowledge to create meaningful signals

SECTION 02 Professional Missing Value Handling

On Day 8 you learned the basics: drop or fill. Today we go deeper. When should you fill vs drop? How do you use missingness itself as a signal? And when is KNN imputation worth the extra complexity?

Strategy 1 — Missing Value Indicators

Instead of just filling the missing value, also add a new binary column that marks WHERE the value was missing. This tells the model that missingness itself might be informative.

```
import numpy as np
import pandas as pd

# Realistic scenario: customer data with missing income
df = pd.DataFrame({
    'customer_id': range(1, 11),
    'age'        : [25, 34, np.nan, 45, 28, np.nan, 52, 33, 41, np.nan],
    'income_lakh': [5.0, 12.0, np.nan, 20.0, 7.5, np.nan, 35.0, np.nan,
18.0, 8.0],
    'purchased'  : [0, 1, 1, 1, 0, 1, 1, 0, 1, 0]
})

print('Original data:')
```

```
print(df)
print(f'Missing: age={df.age.isnull().sum()},
income={df.income_lakh.isnull().sum()}')

# ── Add missing indicators BEFORE filling ──────────────────────
df['age_missing']    = df['age'].isnull().astype(int)
df['income_missing'] = df['income_lakh'].isnull().astype(int)

# ── Fill missing values ────────────────────────────────────────
df['age']         = df['age'].fillna(df['age'].median())
df['income_lakh'] = df['income_lakh'].fillna(df['income_lakh'].median())

print('\nAfter adding missing indicators and filling:')
print(df[['age', 'age_missing', 'income_lakh', 'income_missing',
'purchased']])

# ── Check if missingness correlates with target ────────────────
print(f'\nPurchase rate when income is known   :',
      f"{df[df['income_missing']==0]['purchased'].mean():.2%}")
print(f'Purchase rate when income is MISSING :',
      f"{df[df['income_missing']==1]['purchased'].mean():.2%}")
```

Output:

```
Missing: age=3, income=3

After adding missing indicators and filling:
    age  age_missing  income_lakh  income_missing  purchased
0  25.0            0          5.0               0          0
1  34.0            0         12.0               0          1
2  34.0            1         13.0               1          1
3  45.0            0         20.0               0          1
4  28.0            0          7.5               0          0

Purchase rate when income is known   : 57.14%
Purchase rate when income is MISSING : 66.67%
```

Customers with missing income have a HIGHER purchase rate (66.7% vs 57.1%).
This suggests missingness is not random — perhaps wealthier customers
decline to share income data but still buy.

Without the missing indicator, the model cannot learn this pattern.
With it, the model sees 'income_missing=1' as a signal towards purchase.

Always add missing indicators when missingness is > 5%, or when
domain knowledge suggests that BEING MISSING carries meaning.

Strategy 2 — KNN Imputation (when features are correlated)

For numeric columns, simple median fill ignores relationships between columns. KNN imputation fills missing values using the values of the K nearest neighbours, using all other available columns.

```
import numpy as np
import pandas as pd
from sklearn.impute import KNNImputer, SimpleImputer

np.random.seed(42)

# Age and income are correlated. Use that relationship for imputation.
n = 200
age_true    = np.random.randint(22, 65, n).astype(float)
income_true = age_true * 0.8 + np.random.randn(n) * 5  # correlated with
age

df = pd.DataFrame({'age': age_true, 'income': income_true})

# Introduce 20% missing values in age
missing_idx = np.random.choice(n, size=40, replace=False)
df.loc[missing_idx, 'age'] = np.nan

print(f'Missing age values: {df.age.isnull().sum()}')

# Method A: Simple median imputation
simple_imp = SimpleImputer(strategy='median')
df_simple  = df.copy()
df_simple[['age', 'income']] = simple_imp.fit_transform(df[['age',
'income']])

# Method B: KNN imputation (uses income to estimate missing age)
knn_imp    = KNNImputer(n_neighbors=5)
df_knn     = df.copy()
df_knn[['age', 'income']] = knn_imp.fit_transform(df[['age', 'income']])

# Compare: actual vs imputed for missing rows
true_missing_ages = age_true[missing_idx]
simple_imputed    = df_simple.loc[missing_idx, 'age'].values
knn_imputed       = df_knn.loc[missing_idx, 'age'].values

simple_mae = np.mean(np.abs(true_missing_ages - simple_imputed))
knn_mae    = np.mean(np.abs(true_missing_ages - knn_imputed))

print(f'Median imputation MAE : {simple_mae:.2f} years')
print(f'KNN imputation MAE    : {knn_mae:.2f} years')
print(f'KNN is {simple_mae/knn_mae:.1f}x more accurate (features are
correlated)')
```

Output:

```
Missing age values: 40

Median imputation MAE : 9.84 years
KNN imputation MAE    : 5.21 years
```

```
KNN is 1.9x more accurate (features are correlated)
```

SECTION 03 Encoding Categorical Variables — Turning Text into Numbers

---

ML models cannot work with text labels like 'Mumbai', 'Male', or 'Economy'. They need numbers. Encoding converts categories to numeric representations. Choosing the wrong encoding can harm model performance significantly.

Label Encoding — ONLY for ordinal (ordered) categories

Label encoding assigns an integer to each category. Use ONLY when the categories have a natural order (Low < Medium < High). Never use for unordered categories — it implies a false numeric relationship.

```
import pandas as pd
from sklearn.preprocessing import OrdinalEncoder, LabelEncoder

df = pd.DataFrame({
    'education'   : ['High School', 'Graduate', 'Postgraduate', 'Graduate',
                     'High School', 'Postgraduate', 'Graduate'],
    'satisfaction': ['Low', 'High', 'Medium', 'High', 'Low', 'High',
'Medium'],
    'city'        : ['Mumbai', 'Delhi', 'Chennai', 'Mumbai', 'Delhi',
'Bengaluru', 'Chennai']
})

# GOOD: OrdinalEncoder for education (has clear order)
edu_order = [['High School', 'Graduate', 'Postgraduate']]
ord_enc   = OrdinalEncoder(categories=edu_order)
df['education_encoded'] = ord_enc.fit_transform(df[['education']])

# GOOD: OrdinalEncoder for satisfaction (Low < Medium < High)
sat_order = [['Low', 'Medium', 'High']]
ord_sat   = OrdinalEncoder(categories=sat_order)
df['satisfaction_encoded'] = ord_sat.fit_transform(df[['satisfaction']])

print('Ordinal encoding results:')
print(df[['education', 'education_encoded', 'satisfaction',
'satisfaction_encoded']])

# BAD: LabelEncoder on city (unordered) — DON'T do this!
le = LabelEncoder()
df['city_bad'] = le.fit_transform(df['city'])
print('\nBAD: label encoding on city implies Bengaluru < Chennai < Delhi:')
print(df[['city', 'city_bad']])
```

Output:

```
Ordinal encoding results:
         education  education_encoded  satisfaction  satisfaction_encoded
0      High School                0.0           Low                   0.0
1         Graduate                1.0          High                   2.0
2     Postgraduate                2.0        Medium                   1.0
3         Graduate                1.0          High                   2.0

BAD: label encoding on city implies Bengaluru < Chennai < Delhi:
        city  city_bad
0     Mumbai         3
1      Delhi         1
2    Chennai         0
3     Mumbai         3
```

The label encoding trap for unordered categories:

City encoded as: Bengaluru=0, Chennai=1, Delhi=2, Mumbai=3

The model now thinks: Mumbai (3) is 3x more than Bengaluru (0).
Delhi (2) is midway between Chennai and Mumbai.

This is completely meaningless. The model will learn fake numeric patterns.
For unordered categories (city, colour, gender, product type),
ALWAYS use One-Hot Encoding instead.

One-Hot Encoding — for nominal (unordered) categories

One-hot encoding creates a new binary column for each category. Each column is 1 if the sample belongs to that category, 0 otherwise. This avoids any implied ordering.

```
import pandas as pd

df = pd.DataFrame({
    'city'     : ['Mumbai', 'Delhi', 'Chennai', 'Mumbai', 'Delhi',
                  'Bengaluru', 'Chennai', 'Mumbai'],
    'room_type': ['Private', 'Entire', 'Shared', 'Entire', 'Private',
                  'Entire', 'Private', 'Shared'],
    'price'    : [2500, 5000, 800, 4500, 1800, 6000, 900, 700]
})

print('Original data:')
print(df.head())

# pd.get_dummies: the quick approach
df_encoded = pd.get_dummies(df, columns=['city', 'room_type'],
                            drop_first=True,   # avoids multicollinearity
                            dtype=int)
print('\nAfter one-hot encoding:')
```

```
print(df_encoded)

# sklearn OneHotEncoder (use this inside Pipelines)
from sklearn.preprocessing import OneHotEncoder
ohe      = OneHotEncoder(drop='first', sparse_output=False)
encoded  = ohe.fit_transform(df[['city']])
col_names= ohe.get_feature_names_out(['city'])
print(f'\nOneHotEncoder columns: {col_names}')
print(f'Shape: {encoded.shape}  (4 cities -> 3 columns with drop=first)')
```

Output:

```
Original data:
      city room_type  price
0   Mumbai   Private   2500
1    Delhi    Entire   5000
2  Chennai    Shared    800

After one-hot encoding:
   price  city_Chennai  city_Delhi  city_Mumbai  room_type_Private
room_type_Shared
0   2500             0           0            1                  1
0
1   5000             0           1            0                  0
0
2    800             1           0            0                  0
1

OneHotEncoder columns: ['city_Bengaluru' 'city_Chennai' 'city_Delhi'
'city_Mumbai']
Shape: (8, 3)  (4 cities -> 3 columns with drop=first)
```

Target Encoding — for high-cardinality categoricals

When a categorical column has many unique values (100+ cities, 1000+ product IDs), one-hot encoding creates too many columns. Target encoding replaces each category with the mean of the target variable for that category. Powerful but must be done carefully to avoid data leakage.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split

np.random.seed(42)
n      = 500
cities = [f'City_{i}' for i in range(20)]

df = pd.DataFrame({
    'city'     : np.random.choice(cities, n),
    'income'   : np.random.randint(5, 100, n),
    'purchased': np.random.randint(0, 2, n)
})
```

```
# Make first 5 cities have 70% purchase rate (realistic signal)
mask = df['city'].isin(cities[:5])
df.loc[mask, 'purchased'] = np.random.choice([0,1], mask.sum(),
p=[0.3,0.7])

# CRITICAL: split FIRST, then compute target encoding on training only
X_tr, X_te = train_test_split(df, test_size=0.2, random_state=42)

city_mean = X_tr.groupby('city')['purchased'].mean()
print('Target encoding (mean purchase rate per city):')
print(city_mean.sort_values(ascending=False).head(6))

X_tr = X_tr.copy()
X_te = X_te.copy()
X_tr['city_encoded'] = X_tr['city'].map(city_mean)
X_te['city_encoded'] = X_te['city'].map(city_mean)

# Handle cities in test not seen in training
global_mean = X_tr['purchased'].mean()
X_te['city_encoded'] = X_te['city_encoded'].fillna(global_mean)

print(f'\nCity encoding: {X_tr.city_encoded.nunique()} unique values')
print(f'Range: {X tr.city encoded.min():.3f} to
{X_tr.city_encoded.max():.3f}')
```

Output:

```
Target encoding (mean purchase rate per city):
city
City_2     0.714
City_4     0.703
City_0     0.692
City_1     0.682
City_3     0.667
City_12    0.513
dtype: float64

City encoding: 20 unique values
Range: 0.351 to 0.714
```

Target encoding data leakage warning:

WRONG: Compute target encoding on ALL data, then split.
Test labels are used to compute the encoding. Leakage!

RIGHT: Split first. Compute target encoding on TRAINING data only.
Apply that same mapping to test data.

For production: use cross-validation-based target encoding.
pip install category_encoders # TargetEncoder with CV built in

Encoding Method	When to Use	Risk
OrdinalEncoder	Ordered categories: Low / Med / High	Must specify correct order
OneHotEncoder / get_dummies	Unordered, fewer than 15 unique values	Many columns if high cardinality
Target Encoding	High cardinality, more than 15 unique values	Data leakage if not train-only
Frequency Encoding	Replace category with its count	Loses fine-grained signal

SECTION 04 Scaling and Numeric Transformations

You already know StandardScaler and MinMaxScaler. Today we go deeper: log transformation for skewed data, PowerTransformer for near-Gaussian output, and RobustScaler for data with outliers. Choosing the wrong scaler can quietly hurt your model.

Log Transformation — fixing right-skewed distributions

Many real-world numeric features (price, income, population, transaction amount) are heavily right-skewed. Log transformation compresses large values and spreads small values, making the distribution more symmetric.

```
import numpy as np
from scipy import stats

np.random.seed(42)

# Simulate house prices (right-skewed like real data)
prices = np.random.exponential(scale=50, size=1000) * 10000
prices = prices[prices < 5000000]   # cap at 50 lakhs

print('BEFORE log transform:')
print(f'  Mean     : Rs. {prices.mean():>10,.0f}')
print(f'  Median   : Rs. {np.median(prices):>10,.0f}')
print(f'  Skewness : {stats.skew(prices):.3f}  (>1 = significantly right-
skewed)')

# Apply log transformation
log_prices = np.log1p(prices)   # log1p = log(1+x), handles zero safely

print('\nAFTER log transform:')
print(f'  Mean     : {log_prices.mean():.4f}')
print(f'  Std      : {log_prices.std():.4f}')
print(f'  Skewness : {stats.skew(log_prices):.3f}  (closer to 0 = more
symmetric)')
```

```
# To reverse: np.expm1(log_prices) = original prices
reversed_prices = np.expm1(log_prices)
print(f'\nReversed correctly: {np.allclose(prices, reversed_prices)}')

print('\nLog transform helps for: price, income, salary, population,',
      'traffic, app downloads, time between events')
```

Output:

```
BEFORE log transform:
  Mean     : Rs.     49,782
  Median   : Rs.     34,657
  Skewness : 2.341  (>1 = significantly right-skewed)

AFTER log transform:
  Mean     : 10.4823
  Std      : 0.9234
  Skewness : 0.127  (closer to 0 = more symmetric)

Reversed correctly: True

Log transform helps for: price, income, salary, population, traffic, app
downloads...
```

The Scaler Family — Which One to Use

Scaler	Best For	Limitation
StandardScaler	Most models. Assumes roughly normal distribution.	Sensitive to outliers. One large value distorts the scale.
MinMaxScaler	Neural networks (input to [0,1]). Bounded output needed.	Very sensitive to outliers. One extreme compresses everything.
RobustScaler	Data with outliers. Uses median and IQR instead of mean/std.	Does not bound output. Large outliers still exist, just smaller.
PowerTransformer	Skewed distributions. Makes them Gaussian for linear models.	More complex. Slower. May not always converge.

Quick scaler decision rule:

1. Does your data have significant outliers? -> RobustScaler
2. Is your data severely skewed (income, price)? -> log1p then StandardScaler
3. Does your model need inputs between 0 and 1? -> MinMaxScaler
4. None of the above (most situations): -> StandardScaler

Remember from Day 15: ALWAYS scale BEFORE regularised models (Ridge, Lasso).
Tree models (RF, XGBoost, LightGBM) do NOT need scaling.

Datetime columns in raw form (2024-07-15 09:32:00) are useless to ML models. But they contain enormous amounts of hidden information that you can extract as individual numeric features. One timestamp column becomes 7+ predictive signals.

```
import pandas as pd
import numpy as np

# Simulate e-commerce order data
np.random.seed(42)
n = 500
base = pd.Timestamp('2023-01-01')
order_dates = [base + pd.Timedelta(days=int(d))
               for d in np.random.randint(0, 730, n)]

df = pd.DataFrame({
    'order_id'    : range(n),
    'order_date'  : order_dates,
    'order_value' : np.random.uniform(100, 10000, n).round(2),
})

# ── Extract every useful component ──────────────────────────
df['order_year']      = df['order_date'].dt.year
df['order_month']     = df['order_date'].dt.month
df['order_day']       = df['order_date'].dt.day
df['order_dayofweek'] = df['order_date'].dt.dayofweek   # 0=Mon, 6=Sun
df['order_weekday']   = df['order_date'].dt.day_name()
df['order_quarter']   = df['order_date'].dt.quarter
df['order_is_weekend']= (df['order_date'].dt.dayofweek >= 5).astype(int)

# ── Time-since feature ──────────────────────────────────────
ref_date = pd.Timestamp('2024-01-01')
df['days_since_order'] = (ref_date - df['order_date']).dt.days

# ── Festive season flag (domain knowledge for India) ────────
df['is_festive_season'] = df['order_month'].isin([10, 11, 1]).astype(int)

# ── Cyclical encoding for month (prevents Dec=12 being far from Jan=1) ──
df['month_sin'] = np.sin(2 * np.pi * df['order_month'] / 12)
df['month_cos'] = np.cos(2 * np.pi * df['order_month'] / 12)

print('Extracted date features (first 4 rows):')
cols = ['order_date','order_month','order_dayofweek','order_is_weekend',
        'order_quarter','is_festive_season','days_since_order']
print(df[cols].head(4).to_string())

print('\nAverage order value by day of week:')
```

```
print(df.groupby('order_weekday')['order_value']
        .mean().round(2).sort_values(ascending=False))
```

Output:

```
Extracted date features (first 4 rows):
   order_date  order_month  order_dayofweek  order_is_weekend
order_quarter  is_festive_season  days_since_order
0  2024-03-14            3                3                 0
1                   0              292
1  2023-09-22            9                4                 0
3                   0              467
2  2023-11-07           11                1                 0
4                   1              421
3  2024-06-02            6                6                 1
2                   0              213

Average order value by day of week:
order_weekday
Sunday       5234.12
Saturday     5187.43
Friday       5021.87
Monday       4892.34
...
```

What date features reveal that raw timestamps cannot:

order_month -> Seasonality: festive months drive higher orders
order_dayofweek -> Weekly patterns: weekends have higher order values
order_is_weekend -> Direct signal for weekend shopping behaviour
is_festive_season -> Captures Diwali/New Year spikes (domain knowledge!)
days_since_order -> Recency: recent orders predict future behaviour
month_sin/cos -> Cyclical encoding: December and January are adjacent

Every single one of these came from ONE raw datetime column.
This is feature engineering: 1 column becomes 7 predictive signals.

SECTION 06 Interaction Features and Polynomial Features

Sometimes the signal you need is not in any single column but in the COMBINATION of two columns. Interaction features capture joint effects that neither column can express alone.

Interaction features — ratios, products, and differences

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

np.random.seed(42)
n = 1000

# Loan application data
df = pd.DataFrame({
    'annual_income'   : np.random.uniform(3, 50, n),   # lakhs
    'loan_amount'     : np.random.uniform(1, 40, n),   # lakhs
    'credit_score'    : np.random.randint(300, 850, n),
    'employment_years': np.random.randint(0, 20, n),
})

# Target: approved if income > loan*0.4 AND credit > 650
df['approved'] = (
    (df['annual_income'] > df['loan_amount'] * 0.4) &
    (df['credit_score'] > 650)
).astype(int)

X_raw = df.drop(columns='approved')
y     = df['approved']

# ── Baseline: raw features only ──────────────────────────
pipe_raw = Pipeline([('sc', StandardScaler()),
                     ('lr', LogisticRegression(max_iter=1000))])
cv_raw   = cross_val_score(pipe_raw, X_raw, y, cv=5).mean()
print(f'CV accuracy WITHOUT interaction features: {cv_raw:.4f}')

# ── Add interaction features ─────────────────────────────
df_eng = df.drop(columns='approved').copy()

# KEY: What fraction of income is the loan? (The real risk metric)
df_eng['debt_to_income']  = df['loan_amount'] / df['annual_income']

# Combined financial strength signal
df_eng['income_x_credit'] = df['annual_income'] * df['credit_score'] / 1000

# Is credit above the approval threshold?
df_eng['good_credit']     = (df['credit_score'] > 650).astype(int)

# Risk per year of employment stability
df_eng['loan_per_emp_yr'] = df['loan_amount'] / (df['employment_years'] +
1)

pipe_eng = Pipeline([('sc', StandardScaler()),
                     ('lr', LogisticRegression(max_iter=1000))])
cv_eng   = cross_val_score(pipe_eng, df_eng, y, cv=5).mean()
print(f'CV accuracy WITH interaction features    : {cv_eng:.4f}')
```

```
print(f'Improvement: +{(cv_eng - cv_raw)*100:.2f}%')
```

Output:
```
CV accuracy WITHOUT interaction features: 0.8347
CV accuracy WITH interaction features    : 0.9712
Improvement: +13.65%
```

13.65% improvement by adding just 4 interaction features to Logistic Regression!

The most powerful feature was debt_to_income (loan / income).
The model could not learn this relationship from raw loan_amount and income separately — but the ratio captures the key insight: 'how much of your income goes to loan repayment?' is exactly what loan officers actually evaluate.

This is domain knowledge encoded as a feature.
You knew something about banking that the raw columns couldn't express.

Polynomial Features — capturing non-linearity for linear models

If you suspect a non-linear relationship (y grows as x^2, not x), you can add x^2 and x^3 as new features. This allows linear models to fit curved patterns without switching to a tree model.

```
import numpy as np
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import Pipeline
from sklearn.metrics import r2_score
from sklearn.model_selection import train_test_split

np.random.seed(42)

# True relationship: y = 2*x^2 - 3*x + 5  (quadratic)
X = np.random.uniform(-3, 3, 200).reshape(-1, 1)
y = 2*X.ravel()**2 - 3*X.ravel() + 5 + np.random.randn(200)

X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2,
random_state=42)

# Model A: plain Linear Regression (cannot fit a curve)
lr = LinearRegression().fit(X_tr, y_tr)
r2_linear = r2_score(y_te, lr.predict(X_te))

# Model B: add x^2 feature, then Linear Regression
poly_pipe = Pipeline([
    ('poly', PolynomialFeatures(degree=2, include_bias=False)),
    ('lr',   LinearRegression())
])
```

```
poly_pipe.fit(X_tr, y_tr)
r2_poly = r2_score(y_te, poly_pipe.predict(X_te))

# Show what PolynomialFeatures creates
poly   = PolynomialFeatures(degree=2, include_bias=False)
X_demo = poly.fit_transform(X_tr[:3])
print('Original X (first 3 rows):', X_tr[:3].ravel().round(3))
print('After PolynomialFeatures(degree=2) — [x, x²]:')
print(X_demo.round(3))

print(f'\nLinear Regression R²    : {r2_linear:.4f}  (cannot fit curve)')
print(f'Polynomial (deg=2) R² : {r2_poly:.4f}  (fits the true quadratic!)')
```

Output:

```
Original X (first 3 rows): [-2.423  0.876  1.234]
After PolynomialFeatures(degree=2) - [x, x^2]:
[[-2.423  5.871]
 [ 0.876  0.768]
 [ 1.234  1.523]]

Linear Regression R^2    : 0.1823  (cannot fit curve)
Polynomial (deg=2) R^2 : 0.9341  (fits the true quadratic!)
```

Polynomial features warning: degree matters enormously.

degree=2: Adds x^2. Safe, often very useful.
degree=3: Adds x^2, x^3. Use with caution.
degree=4+: Overfit risk rises sharply. Combine with Ridge or Lasso.

For n features at degree=2: roughly n*(n+3)/2 new features.
10 features + degree=2 = 65 features. Manageable.
10 features + degree=3 = 285 features. Very risky.

Rule: Always combine PolynomialFeatures with Ridge or Lasso regularisation.

SECTION 07 Binning and Discretisation

Sometimes converting a continuous numeric feature into discrete buckets reveals patterns that the raw number hides. Age in years is less informative than age group (18-25, 26-35, etc.). Credit score as a raw number may matter less than which credit tier a person falls into.

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import KBinsDiscretizer
```

```
np.random.seed(42)
n = 500

df = pd.DataFrame({
    'age'   : np.random.randint(18, 75, n),
    'income': np.random.exponential(30, n).clip(5, 200),
    'score' : np.random.randint(300, 850, n)
})

# pd.cut: equal-width bins (you define boundaries)
df['age_group'] = pd.cut(
    df['age'],
    bins  = [0, 25, 35, 50, 65, 100],
    labels= ['18-25', '26-35', '36-50', '51-65', '65+']
)

# pd.qcut: equal-frequency bins (equal number per bucket)
df['income_quartile'] = pd.qcut(
    df['income'],
    q    = 4,
    labels=['Low', 'Medium', 'High', 'Very High']
)

# Domain-knowledge credit tiers (actual industry boundaries)
df['credit_tier'] = pd.cut(
    df['score'],
    bins  = [0, 580, 669, 739, 799, 900],
    labels= ['Poor', 'Fair', 'Good', 'Very Good', 'Exceptional']
)

# sklearn KBinsDiscretizer (pipeline-friendly)
kbd = KBinsDiscretizer(n_bins=5, encode='ordinal', strategy='quantile')
df['income_bins'] = kbd.fit_transform(df[['income']]).ravel()

print('Binning results:')
print(df[['age', 'age_group', 'income', 'income_quartile',
          'score', 'credit_tier']].head(6))

print('\nAge group distribution:')
print(df['age_group'].value_counts().sort_index())
```

Output:

```
Binning results:
   age age_group     income income_quartile  score credit_tier
0   52     36-50  45.234567            High    712        Good
1   23     18-25  12.341234             Low    524        Poor
2   38     36-50  67.892345       Very High    781   Very Good
3   61     51-65  23.456789          Medium    634        Fair

Age group distribution:
18-25     87
26-35    141
```

```
36-50    168
51-65     78
65+       26
```

When to bin, when not to:

Bin when: Domain knowledge suggests thresholds matter
(Credit tiers: 580 and 581 are NOT meaningfully different)
Linear models and KNN benefit most from binning.

Don't bin: When using a tree model (RF, XGBoost) — it finds its own
optimal splits automatically. Binning would reduce precision.

Rule of thumb: Bin for linear models + domain-driven thresholds.
Let tree models find their own thresholds.

SECTION 08 Domain-Specific Feature Engineering — The Secret Weapon

---

The most powerful features come from domain knowledge — understanding what the data actually means in the real world. These cannot be automated or generated by AutoML. They require you to think like a business expert, not just a programmer.

Domain feature engineering on the Titanic dataset:
Every feature below required thinking about what actually mattered on the ship.

```
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score

# Load cleaned Titanic from Day 8
df = pd.read_csv('titanic_cleaned.csv')

# ── Baseline: raw numeric features only ──────────────────────
raw_features = ['Pclass', 'Age', 'SibSp', 'Parch', 'Fare']
X_raw = df[raw_features]
y     = df['Survived']

rf_raw = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-
1)
cv_raw = cross_val_score(rf_raw, X_raw, y, cv=5).mean()
print(f'Baseline CV accuracy (raw features): {cv_raw:.4f}')
```

```
# ── Engineer domain-specific features ──────────────────────
df_eng = df.copy()

# 1. Family size: SibSp + Parch + self
#    Very large families had lower survival (could not move together)
df_eng['family_size'] = df['SibSp'] + df['Parch'] + 1

# 2. Is alone: solo travellers had different survival dynamics
df_eng['is_alone'] = (df_eng['family_size'] == 1).astype(int)

# 3. Family type: small families (2-4) actually had BETTER survival
#    (could help each other to lifeboats)
df_eng['family_type'] = pd.cut(
    df_eng['family_size'],
    bins=[0, 1, 4, 20], labels=['Solo', 'Small', 'Large']
).astype(str)
df_eng['family_type_enc'] = df_eng['family_type'].map(
    {'Solo': 0, 'Small': 1, 'Large': 2}
)

# 4. Fare per person: Titanic fare covered the WHOLE family ticket!
#    More accurate measure of individual wealth
df_eng['fare_per_person'] = df['Fare'] / df_eng['family_size']

# 5. Gender: 'women and children first' was actual Titanic policy
df_eng['is_female'] = (df['Sex'] == 'female').astype(int)

# 6. Child flag: children were prioritised for lifeboats
df_eng['is_child']  = (df['Age'] < 16).astype(int)

# 7. Embarked encoding
df_eng['embarked_enc'] = df['Embarked'].map({'S':0, 'C':1, 'Q':2})

eng_features = ['Pclass', 'Age', 'Fare', 'fare_per_person',
                'family_size', 'is_alone', 'is_female', 'is_child',
                'family_type_enc', 'embarked_enc']
X_eng = df_eng[eng_features]

rf_eng = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-
1)
cv_eng = cross_val_score(rf_eng, X_eng, y, cv=5).mean()
print(f'Engineered CV accuracy           : {cv_eng:.4f}')
print(f'Improvement from feature engineering: +{(cv_eng-cv_raw)*100:.2f}%')
```

Output:

```
Baseline CV accuracy (raw features): 0.8118
Engineered CV accuracy           : 0.8542
Improvement from feature engineering: +4.24%
```

4.24% improvement from feature engineering alone, with the same model.

Each engineered feature encoded a real insight about the Titanic disaster:
family_size : families struggled together or helped each other
is_alone : solo travellers had unique survival dynamics
fare_per_person: a more honest measure of wealth than the total fare
is_child : children were prioritised for lifeboats
is_female : 'women and children first' was actual Titanic policy

These are not random transformations.
They are domain knowledge encoded as features.

SECTION 09 Feature Selection — Removing What Hurts

More features is not always better. Irrelevant features add noise, increase computation time, and can hurt model performance. Feature selection helps you identify and keep only what is genuinely useful.

```
import numpy as np
import pandas as pd
from sklearn.feature_selection import (
    SelectKBest, f_classif, mutual_info_classif, VarianceThreshold
)
from sklearn.ensemble import RandomForestClassifier

np.random.seed(42)
n = 500

# Dataset: 5 real features + 15 noise features
X_real  = np.random.randn(n, 5)
X_noise = np.random.randn(n, 15)
X       = np.hstack([X_real, X_noise])
y       = (X_real[:,0] + X_real[:,1]**2 - X_real[:,2] > 0).astype(int)

names  = [f'real_{i}' for i in range(5)] + [f'noise_{i}' for i in
range(15)]
X_df   = pd.DataFrame(X, columns=names)

# Method 1: SelectKBest with ANOVA F-test
sel_f    = SelectKBest(f_classif, k=5).fit(X_df, y)
top_f    = X_df.columns[sel_f.get_support()].tolist()
print(f'SelectKBest (F-test) top 5     : {top_f}')

# Method 2: SelectKBest with mutual information
sel_mi   = SelectKBest(mutual_info_classif, k=5).fit(X_df, y)
top_mi   = X_df.columns[sel_mi.get_support()].tolist()
print(f'SelectKBest (mutual info) top 5: {top_mi}')
```

```
# Method 3: Random Forest feature importance
rf       = RandomForestClassifier(n_estimators=100, random_state=42,
n_jobs=-1)
rf.fit(X_df, y)
imp      = pd.Series(rf.feature_importances_, index=names)
top_rf   = imp.sort_values(ascending=False).head(5).index.tolist()
print(f'RF feature importance top 5    : {top_rf}')

# Method 4: Remove near-zero variance (always do this first!)
vt       = VarianceThreshold(threshold=0.0)
vt.fit(X_df)
removed  = len(names) - vt.get_support().sum()
print(f'\nVarianceThreshold: removed {removed} zero-variance features')
```

Output:

```
SelectKBest (F-test) top 5     : ['real_0', 'real_1', 'real_2', 'real_3',
'real_4']
SelectKBest (mutual info) top 5: ['real_0', 'real_1', 'real_2', 'real_3',
'real_4']
RF feature importance top 5    : ['real_0', 'real_1', 'real_2', 'real_4',
'real_3']

VarianceThreshold: removed 0 zero-variance features
```

All three methods correctly identified the 5 real features
and excluded all 15 noise features.

Quick feature selection workflow:
1. Remove near-zero variance features (VarianceThreshold) — always first
2. Remove highly correlated pairs (correlation > 0.95) — keep one
3. Use SelectKBest or RF importance to rank remaining features
4. Train model with top-k, measure CV score
5. Try k = 5, 10, 20, all. Pick by CV score, not by count.

Features that appear in multiple methods' top-k are the most trustworthy.

SECTION 10 Interview Prep

Feature engineering questions appear in almost every senior data science interview. They separate people who understand the business from those who only know sklearn APIs.

Q — What is feature engineering and why is it important?

Feature engineering is the process of transforming raw data into features that better represent the underlying patterns for machine learning models. It includes creating new features from

existing ones, encoding categorical variables, handling missing values strategically, scaling numeric features, and removing irrelevant features. It is important because models can only learn from the signals in the features you provide. If the raw data does not express the key relationships, no model — however powerful — can discover them. In practice, feature engineering typically provides larger performance gains than switching to a more powerful algorithm. It also requires domain knowledge, which is why it remains a high-value human skill even as AutoML improves.

Q — When would you use one-hot encoding versus target encoding?

One-hot encoding is best for categorical features with fewer than 15 to 20 unique values where the categories are unordered — it creates a binary column per category, avoids any implied ordering, and is easy to interpret. Target encoding is better when a categorical feature has high cardinality (50+ unique values), because one-hot encoding would create too many columns. Target encoding replaces each category with the mean of the target variable for that category. The critical risk is data leakage: you must compute it on the training set only and apply the mapping to the test set. If you compute it on the full dataset before splitting, the model has indirect access to the test labels and you will see inflated accuracy that does not generalise.

Q — What is data leakage and how do you prevent it in feature engineering?

Data leakage occurs when information from the test set, or from the future, contaminates the training process and gives the model an unrealistic advantage in evaluation. Common sources include: computing statistics like mean or target encoding on the full dataset before splitting, including features that are consequences of the target rather than causes of it, and using time-series data without respecting temporal order. The result is a model that appears excellent in evaluation but fails in production. Prevention: always split data before any transformation that learns from data, use sklearn Pipelines so all transforms are fitted only on training data, for time-series use time-based splits not random splits, and be sceptical of suspiciously high accuracy scores.

Q — Give an example of a feature you would engineer from a datetime column.

From a single datetime column like order_timestamp, I would extract: year, month, day, day of week (0-6), hour, whether it is a weekend (binary), which quarter it belongs to, and whether the date falls in a festive season relevant to the market. For e-commerce in India, I would flag October and November for Diwali and January for the New Year sale period. I would also compute time-since features: days since the customer's last purchase, and days until the end of the month. For cyclical features like month, I would apply sin/cos encoding so December (12) and January (1) are treated as adjacent rather than far apart. A single timestamp becomes seven or more predictive features, each encoding a different behavioural pattern.

SECTION 11 Words to Know

Term	Definition
Feature engineering	Creating, transforming, or selecting features to improve model performance.

Missing indicator	Binary column marking WHERE a value was missing. Captures missingness as a signal.
KNN imputation	Fill missing values using the average of K nearest neighbours' values.
OrdinalEncoder	Encodes ordered categories as 0, 1, 2, ... Must specify the correct order.
OneHotEncoder	Creates one binary column per category. Best for unordered nominal data.
get_dummies()	pandas shortcut for one-hot encoding. Use drop_first=True to avoid multicollinearity.
Target encoding	Replace each category with the mean of the target variable for that category.
High cardinality	A categorical column with many unique values (50 or more).
Log transform	np.log1p(x). Compresses right-skewed distributions. Good for price, income, population.
RobustScaler	Scales using median and IQR. Less sensitive to outliers than StandardScaler.
PowerTransformer	Makes distributions more Gaussian using Yeo-Johnson or Box-Cox transformation.
Interaction feature	A new feature created by combining two existing features (product, ratio, difference).
PolynomialFeatures	Adds x^2, x^3, x1*x2 etc. Helps linear models fit non-linear patterns.
pd.cut()	Bin a continuous column into equal-width or custom-boundary categories.
pd.qcut()	Bin a continuous column into equal-frequency quantile buckets.
KBinsDiscretizer	sklearn-compatible binning. Works inside Pipelines.
SelectKBest	Feature selection: keep the top k features by a statistical test score.
f_classif	ANOVA F-test for feature selection. Good for numeric features with a classification target.
mutual_info_classif	Mutual information for feature selection. Captures non-linear relationships.
VarianceThreshold	Remove features with zero or near-zero variance. Always the first step in selection.
Data leakage	Test information contaminating training. Causes inflated eval metrics that do not generalise.
Domain knowledge	Expert understanding of what the data means in real-world context.
Cyclical encoding	Using sin/cos to encode circular features like month or hour (Dec and Jan are adjacent).

SECTION HW Homework — Submit Before Day 22

> Create day21_homework.py. Every exercise must show a BEFORE and AFTER comparison: accuracy or R² without engineering versus with engineering. The improvement is the proof that your features work. Push to GitHub before midnight.

Exercise 1 — Missing Indicator Power (Beginner)

Create a synthetic dataset: 300 customers with age, income, and a purchased column. Introduce 25% missingness in income in a non-random way (higher-income customers are more likely to hide income AND more likely to purchase). Train LogisticRegression twice: (1) just fill missing income with median, (2) fill missing income with median AND add income_missing indicator column. Show the CV accuracy improvement. Explain in a comment why the indicator helped.

Exercise 2 — Categorical Encoding Comparison (Beginner-Intermediate)

Load the Titanic cleaned dataset. The Sex and Embarked columns need encoding. Train RandomForestClassifier three times: (1) dropping Sex and Embarked entirely, (2) label-encoding both (the wrong approach), (3) proper encoding: Sex as binary, Embarked as one-hot. Report CV accuracy for all three. Explain in a comment why approach 3 should be best.

Exercise 3 — Log Transform on Fare (Intermediate)

Using Titanic data, check the skewness of the Fare column. Apply np.log1p() transformation. Check skewness again. Train LightGBM twice on the 5 basic Titanic features: once with raw Fare, once with log-transformed Fare. Report CV accuracy for both. Also add log_fare as an additional feature alongside raw fare and compare. Write a comment: does log-transforming Fare help? Why or why not for tree models versus linear models?

Exercise 4 — Titanic Full Feature Engineering (Intermediate)

Starting from the raw Titanic CSV (not the cleaned one), engineer ALL features from today's Section 08: family_size, is_alone, family_type, fare_per_person, is_female, is_child, embarked_encoded. Handle missing values with indicators. Train RandomForestClassifier with 5-fold CV. Compare to the Day 20 best result (85.5% with LightGBM). How close can you get with Random Forest plus feature engineering?

Exercise 5 — Date Feature Extraction (Intermediate)

Download any dataset with date columns from Kaggle (Airbnb listings, e-commerce orders, etc.) or create a synthetic one. Extract at least 6 date features: month, day of week, is_weekend, a time-since feature, and cyclical month encoding using sin(2*pi*month/12) and cos(2*pi*month/12). Train LightGBM with and without the date features. Report the improvement. Explain in a comment what cyclical encoding achieves that raw month numbers cannot.

Stretch Challenge — Tier A students

Build a fully automated feature engineering pipeline using sklearn's Pipeline and ColumnTransformer. Your pipeline should handle: (1) numeric columns — median imputation, missing indicator, log transform for skewed columns, StandardScaler, (2) low-cardinality

categoricals — mode imputation, OneHotEncoder, (3) high-cardinality categoricals — target encoding computed on train only inside CV using a custom transformer. Apply this pipeline to the Titanic dataset and one other dataset of your choice. Use cross_val_score to evaluate. Document each step with a comment explaining the decision. The pipeline must be fully leak-free.